

# Retail Sales Data Pipeline

*Outcome: A production-shaped medallion lakehouse that lands POS and service data, enforces fail-closed data quality, publishes six gold marts, and feeds Tableau sales and service workbooks — with Fake Store API ingest and HTTP sinks for downstream systems.*

## 1. Why

Retail operators need a single, trusted path from store transactions and customer-service tickets to executive sales and SLA reporting. Spreadsheet extracts and ad-hoc SQL leave gaps: no quarantine trail, no referential checks, and no repeatable mart grain for BI. This system exists to give analytics and operations teams a governed bronze → silver → gold pipeline that can run locally or on a cluster, orchestrate on a daily schedule, and hand off clean marts to Tableau and warehouse SQL.

## 2. What — Deliverables

- **Medallion lakehouse** — bronze landing with ingest metadata; silver dims/facts with quarantine; six gold business marts.
- **API source** — Fake Store HTTP pull (products, carts, users) into bronze `api_*` tables.
- **API sinks** — POST gold summary to local FastAPI `/ingest` and JSONPlaceholder `/posts`.
- **Data quality** — 12 silver DQ checks (PK uniqueness, nulls, positive amounts, FK integrity, channel/payment enums) with fail-on-error.
- **Orchestration** — Apache Airflow DAG: `extract_api` → raw refresh → bronze → silver+DQ → gold → sink → warehouse hint.
- **Warehouse SQL** — Snowflake-flavored schemas, dimensions, facts, and gold CTAS/views under `sql/`.
- **Tableau pack** — Sales Performance and Customer Satisfaction & Service workbooks (overview, stores/regions/trends; SLA, pending/aging).
- **Cloud mapping** — ADF / Databricks / ADLS / Fabric design notes for enterprise handoff.

## 3. When — Delivery Plan

A realistic engagement for a mid-size retail analytics build typically spans four phases:

| Phase    | Focus                                                                             | Typical duration |
|----------|-----------------------------------------------------------------------------------|------------------|
| Design   | Source contracts, medallion grain, DQ rules, mart specs, BI page briefs           | 3–5 days         |
| Build    | Bronze/silver/gold jobs, API source & sinks, Airflow DAG, Snowflake SQL           | 1.5–2.5 weeks    |
| Validate | DQ gate dry-runs, quarantine audit, Tableau rebuild from mart CSVs, sink receipts | 3–5 days         |
| Handoff  | Runbooks, cloud mapping, env templates, acceptance walkthrough                    | 2–3 days         |

Total calendar time for a comparable scoped delivery: roughly **3–5 weeks**, depending on source system access and BI review cycles.

## 4. How — Architecture & Stack

Transforms use Python with a **PySpark job structure** and a **pandas fallback** (default for local/CI). Layer boundaries and DQ checks stay identical regardless of engine. Configuration is YAML-driven (*config/pipeline.yaml*); secrets stay out of the repo (*.env.example* names only).

| Concern       | Choice                                                   |
|---------------|----------------------------------------------------------|
| Architecture  | Medallion — bronze / silver / gold                       |
| Transforms    | Python + PySpark structure; pandas engine for local runs |
| Orchestration | Apache Airflow (daily 06:00 UTC schedule in config)      |
| Warehouse     | Snowflake-flavored DDL + marts                           |
| Quality       | Custom DQ with fail_on_error; quarantine path            |
| BI            | Tableau workbooks + screenshots                          |
| API source    | Fake Store API (no key)                                  |
| API sinks     | Local FastAPI landing + JSONPlaceholder                  |

## 5. Architecture Diagram

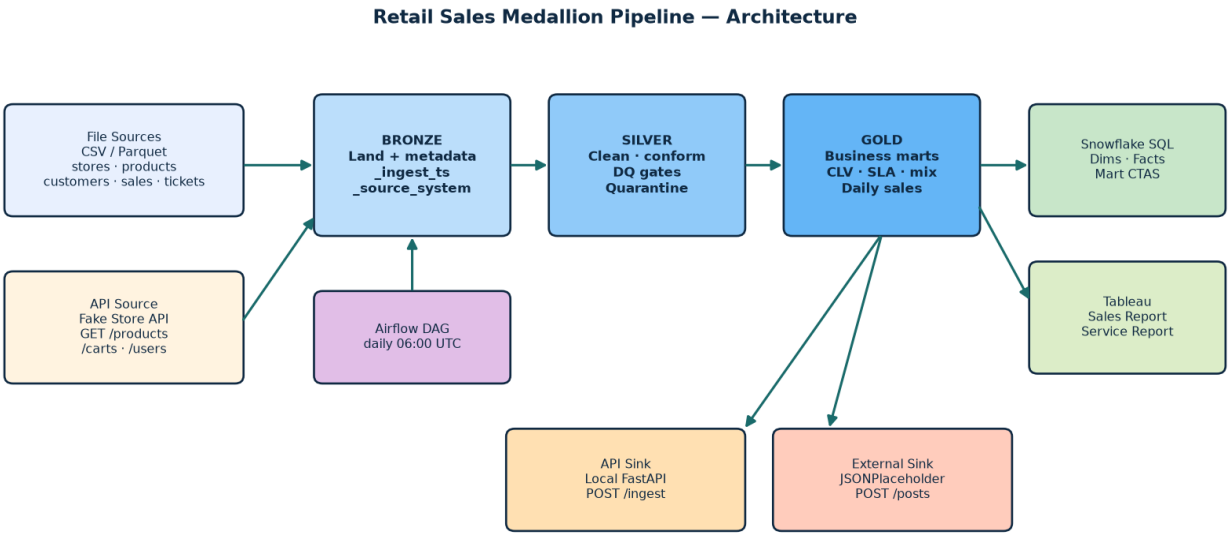


Figure 1. End-to-end flow: file + API sources → medallion layers → Tableau / SQL / HTTP sinks.

## 6. Technical Depth

### Observed pipeline volumes (bronze → gold run)

| Layer / artifact                       | Rows / result  |
|----------------------------------------|----------------|
| Bronze — stores / products / customers | 25 / 120 / 500 |

| Layer / artifact                              | Rows / result                 |
|-----------------------------------------------|-------------------------------|
| Bronze — sales_transactions / service_tickets | 5,000 / 1,800                 |
| Silver — fact sales (post-DQ)                 | 4,985                         |
| Silver — quarantine_sales (auditable rejects) | 15                            |
| Silver DQ checks                              | 12 / 12 passed                |
| Gold — mart_daily_sales_by_store              | 3,782                         |
| Gold — mart_daily_sales_by_category           | 1,721                         |
| Gold — mart_customer_lifetime_value           | 479                           |
| Gold — mart_product_performance               | 113                           |
| Gold — mart_channel_mix                       | 3,276                         |
| Gold — mart_service_performance               | 1,800                         |
| Gold DQ checks                                | 6 / 6 passed                  |
| Service KPIs                                  | 1,800 tickets · avg CSAT 3.87 |

Silver DQ covers fact non-empty, unique *transaction\_id*, required non-nulls, positive quantity/net amount, store & product FK integrity, and allowed channel/payment enums. Invalid rows land in *quarantine\_sales* instead of silent drops. Top gold revenue categories in the published summary include Sports (~\$1.19M), Apparel (~\$1.05M), and Electronics (~\$1.00M).

Key design choices: fail-closed DQ, engine switch (*pandas|spark|auto*) without changing job graph, and SQL/cloud docs that bridge lakehouse code to warehouse and Azure/Fabric delivery.

## 7. Outcomes — What the Client Gets

- A runnable medallion pipeline with documented layers, manifests, and DQ reports.
- Six gold marts ready for Tableau and Snowflake-style warehouse load.
- Two Tableau report packs (Sales; Satisfaction & Service) with page briefs and LOD calc notes.
- Working Fake Store API source stage and dual HTTP sinks with landing receipts.
- Airflow DAG task graph for scheduled production operation.
- Cloud mapping guidance for ADF, Databricks, ADLS, and Microsoft Fabric.
- One-command local rebuild: *python scripts/run\_local.py --engine pandas*.

## 8. Repository

<https://github.com/user-JB007/retail-sales-pipeline>

License: MIT. Configuration and architecture details live under *config/*, *docs/architecture.md*, and *docs/cloud\_mapping.md*.